

Private Network Service Platform

Phase 1 Architecture & Evidence Report

Members:

- Vaageesh Kumar Singh – 2401020073
 - Shreyash Golhani – 2401020069
 - Ananya Narang – 2401020087
 - Aman Kumar – 2401010061
- Course: Computer Networks

September 29, 2026

Contents

1	Introduction	2
2	Network Architecture and Topology	2
2.1	Machine Roles	2
2.2	Request Flow	2
2.3	IP and Service Inventory	2
3	Task A: Establishing the Private LAN	3
4	Task B: Private DNS Server Configuration	3
5	Task C: Backend REST Services	5
5.1	Backend A (Mac 3) – Python <code>http.server</code>	5
5.2	Backend B (Mac 4) – FastAPI + Uvicorn	6
5.3	Endpoint Contract	6
6	Task D: Edge Reverse Proxy and Load Balancing	7
6.1	Load Balancing Verification	7
7	Task E: HTTPS / TLS Configuration	8
7.1	TLS Handshake Evidence (Wireshark)	8
8	Task F: HTTP Caching Behaviour	8
9	Task G: Complete Protocol Flow Capture	10
9.1	DNS Layer	10
9.2	TCP Handshake	10
9.3	TLS Handshake	10
9.4	HTTP Headers	10
9.5	Port Identification Summary	10
10	Conclusion and Learning Summary	10

1 Introduction

This report documents the design, implementation, and verification of a private, fully local network service platform built as part of the Computer Networks course project. The system allows a client machine to resolve a private domain name through a team-operated DNS server, connect to a reverse proxy over HTTPS, and receive a load-balanced response from one of two backend application servers. Every stage of this request path – DNS resolution, TCP connection setup, TLS handshake, and HTTP response – was captured and verified using `dig`, `curl`, and Wireshark.

The guiding principle of the project was that *the application stays simple; the network is the project*. Accordingly, the backend applications are minimal REST services, and the engineering effort is concentrated on DNS configuration, reverse proxying, load balancing, and TLS termination.

2 Network Architecture and Topology

2.1 Machine Roles

The team’s environment consists of four macOS laptops on a shared private Wi-Fi network (SSID: `Rishhood.Learner`), each assigned a distinct network role.

Machine	Primary Role	Services Running	Represents
Mac 1	Private DNS Server + Test Client	dnsmasq, dig/nslookup, curl	Managed DNS (Route 53)
Mac 2	Edge / Reverse Proxy + Load Balancer	nginx, TLS certificate, upstream config	Cloud load balancer / CDN edge
Mac 3	Backend Server A	Python <code>http.server</code> REST app (port 3001)	Application server instance A
Mac 4	Backend Server B + Test Client	FastAPI + Uvicorn REST app (port 3002), curl/browser	Application server instance B

Table 1: Machine roles in the private network platform

2.2 Request Flow

A client request travels through the system as follows:

Client (Mac 1 / Mac 4) → DNS Query (Mac 1) → HTTPS Request (Mac 2 / nginx) → Backend A (Mac 3) or Backend B (Mac 4)

The private domain used throughout the project is:

`app.team.cn.test`

2.3 IP and Service Inventory

Machine	Role	Private IPv4	Port(s)
Mac 1	DNS Server (dnsmasq)	10.7.20.217	53 (UDP)
Mac 2	Edge / nginx	10.7.20.207	443 (HTTPS), 80 (redirect, if used)
Mac 3	Backend A	10.7.9.120	3001
Mac 4	Backend B	10.7.10.70	3002

Table 2: IP address and port inventory.

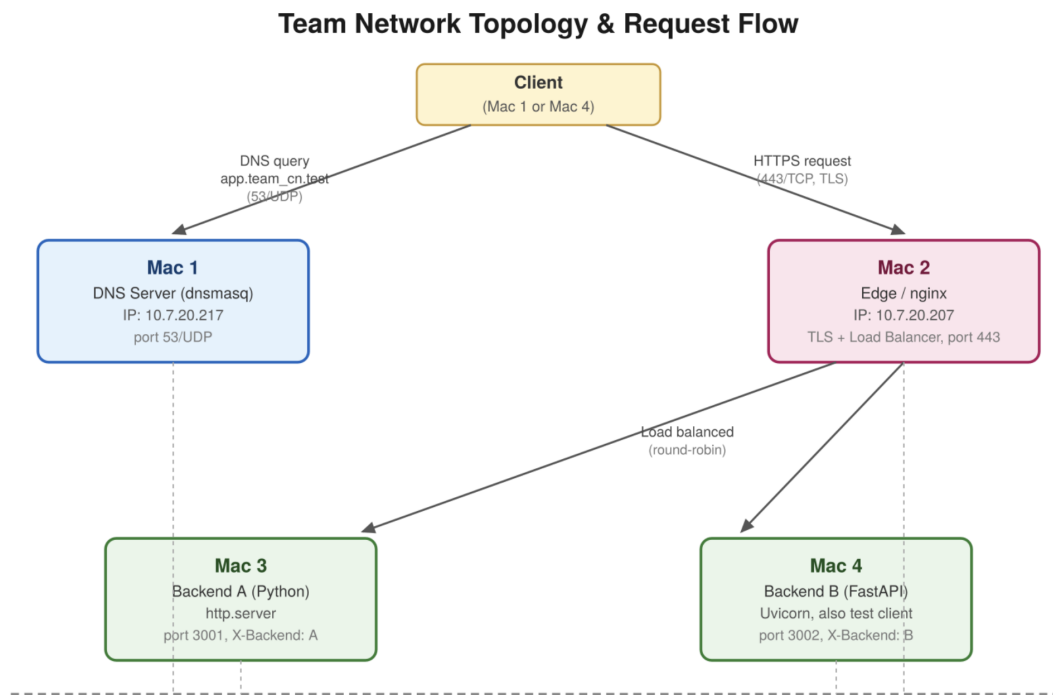


Figure 1: Team network topology and request flow

3 Task A: Establishing the Private LAN

All four machines were connected to the same private Wi-Fi network (`RishhoodLearner`). Reachability between every pair of machines was verified using `ping`.

```

1 ping -c 4 10.7.20.217 # Mac 1 (DNS)
2 ping -c 4 10.7.20.207 # Mac 2 (Edge/nginx)
3 ping -c 4 10.7.9.120 # Mac 3 (Backend A)
4 ping -c 4 10.7.10.70 # Mac 4 (Backend B)

```

Listing 1: Reachability verification

Each ping test returned 0.0% packet loss, confirming Layer 2/3 connectivity across all team machines before any higher-layer services were configured.

4 Task B: Private DNS Server Configuration

Mac 1 was configured to run `dnsmasq` as the team's private DNS resolver. The following DNS records were created:

```

Last login: Mon Sep 28 20:15:08 on ttys001
Vaageesh@Vaageeshs-MacBook-Pro ~ % ping 10.7.20.207
PING 10.7.20.207 (10.7.20.207): 56 data bytes
64 bytes from 10.7.20.207: icmp_seq=0 ttl=64 time=17.406 ms
64 bytes from 10.7.20.207: icmp_seq=1 ttl=64 time=51.174 ms
64 bytes from 10.7.20.207: icmp_seq=2 ttl=64 time=90.149 ms
64 bytes from 10.7.20.207: icmp_seq=3 ttl=64 time=135.504 ms
64 bytes from 10.7.20.207: icmp_seq=4 ttl=64 time=7.708 ms
64 bytes from 10.7.20.207: icmp_seq=5 ttl=64 time=51.070 ms
64 bytes from 10.7.20.207: icmp_seq=6 ttl=64 time=88.284 ms
64 bytes from 10.7.20.207: icmp_seq=7 ttl=64 time=11.064 ms
64 bytes from 10.7.20.207: icmp_seq=8 ttl=64 time=6.869 ms
64 bytes from 10.7.20.207: icmp_seq=9 ttl=64 time=6.975 ms
64 bytes from 10.7.20.207: icmp_seq=10 ttl=64 time=10.782 ms

```

Figure 2: Ping connectivity test from client to Mac 2 (edge), 0% packet loss

Record	Resolves to
app.team_cn.test	10.7.20.207 (Mac 2)
api.team_cn.test	10.7.20.207 (Mac 2)

Client machines (Mac 2 and Mac 4, at minimum) had their DNS resolver set to 10.7.20.217 via **System Settings** → **Wi-Fi** → **Details** → **DNS**, replacing the router-assigned DNS server.

```

1 dig app.team_cn.test
2 nslookup app.team_cn.test

```

Listing 2: DNS resolution verification

Resolution was confirmed to return Mac 2's IP address (10.7.20.207), and the final application was always accessed by domain name – never by directly entering the IP address – satisfying the project's requirement to distinguish DNS resolution from the subsequent TCP/HTTPS connection.

```

Vaageesh@Vaageeshs-MacBook-Pro ~ % dig app.test_cn

; <<> DiG 9.10.6 <<> app.test_cn
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NXDOMAIN, id: 56264
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;app.test_cn.                IN      A
;
;; AUTHORITY SECTION:
.                86400   IN      SOA      a.root-servers.net. nstld.verisign-grs.com. 2026092900 1800 900 604800 86400

;; Query time: 36 msec
;; SERVER: 10.7.20.217#53(10.7.20.217)
;; WHEN: Tue Sep 29 18:46:19 IST 2026
;; MSG SIZE rcvd: 115

Vaageesh@Vaageeshs-MacBook-Pro ~ %

```

Figure 3: Successful DNS resolution of app.team_cn.test to Mac 2's IP (10.7.20.207)

5 Task C: Backend REST Services

Two independent backend REST services were implemented, each identifying itself via an X-Backend response header so that load-balanced responses can be distinguished.

5.1 Backend A (Mac 3) – Python http.server

Backend A listens on all interfaces (0.0.0.0) on port 3001 so that it is reachable from other machines on the LAN, and not just localhost.

```

1 from http.server import BaseHTTPRequestHandler, HTTPServer
2 import json
3
4 class BackendAHandler(BaseHTTPRequestHandler):
5     def send_json(self, data, status=200):
6         body = json.dumps(data).encode("utf-8")
7
8         self.send_response(status)
9
10        self.send_header("Content-Type", "application/json")
11        self.send_header("Content-Length", str(len(body)))
12
13        # Required for the project
14        self.send_header("X-Backend", "A")
15
16        # Useful for Phase 1 caching demonstration
17        self.send_header("Cache-Control", "max-age=60")
18
19        self.end_headers()
20        self.wfile.write(body)
21
22    def do_GET(self):
23        if self.path == "/":
24            self.send_json({
25                "backend": "A",
26                "message": "Backend A is running"

```

```

27         })
28
29         elif self.path == "/api/status":
30             self.send_json({
31                 "backend": "A",
32                 "status": "ok"
33             })
34
35         else:
36             self.send_json({
37                 "error": "Not Found"
38             }, status=404)
39
40     def log_message(self, format, *args):
41         print(f"[Backend A] {self.address_string()} - {format % args}")

```

Listing 3: Backend A implementation (backend_a.py)

5.2 Backend B (Mac 4) – FastAPI + Uvicorn

Backend B implements the same endpoint contract using FastAPI, run with Uvicorn bound to 0.0.0.0:3002.

```

1 from fastapi import FastAPI
2 from fastapi.responses import JSONResponse
3
4 app = FastAPI()
5
6 @app.get("/")
7 def home():
8     response = JSONResponse({
9         "backend": "B",
10        "status": "ok",
11        "message": "Backend B is running"
12    })
13    response.headers["X-Backend"] = "B"
14    response.headers["Cache-Control"] = "max-age=60"
15    return response
16
17 @app.get("/api/status")
18 def status():
19     response = JSONResponse({
20         "backend": "B",
21         "status": "ok"
22     })
23     response.headers["X-Backend"] = "B"
24     return response

```

Listing 4: Backend B implementation (main.py)

```

1 uvicorn main:app --host 0.0.0.0 --port 3002

```

Listing 5: Launch command for Backend B

5.3 Endpoint Contract

Endpoint	Expected Response
GET /	JSON/HTML confirming the service is running
GET /api/status	{"backend": "A"/"B", "status": "ok"}
Response header	X-Backend: A or X-Backend: B on every response

6 Task D: Edge Reverse Proxy and Load Balancing

Mac 2 runs `nginx` as the single public entry point for the system. Clients never connect directly to Mac 3 or Mac 4; `nginx` forwards each request to one of the two backends using round-robin load balancing.

```

1 upstream backend_pool {
2     server 10.7.10.XX:3001;    # Mac 3 - Backend A
3     server 10.7.10.YY:3002;    # Mac 4 - Backend B
4 }
5
6 server {
7     listen 443 ssl;
8     server_name app.team.cn.test;
9
10    ssl_certificate      /path/to/app.team.cn.test.crt;
11    ssl_certificate_key  /path/to/app.team.cn.test.key;
12
13    location / {
14        proxy_pass http://backend_pool;
15        proxy_set_header Host $host;
16        proxy_set_header X-Real-IP $remote_addr;
17    }
18 }
```

Listing 6: `nginx` upstream and load balancing configuration (edit IPs to match your team)

6.1 Load Balancing Verification

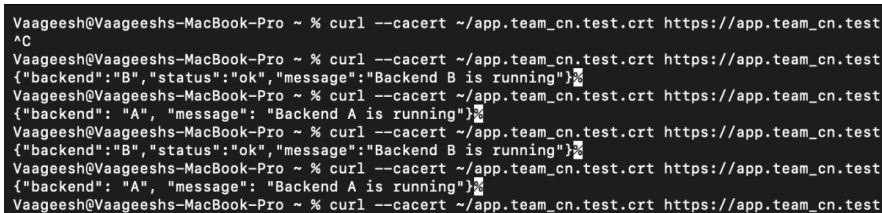
Repeated HTTPS requests to `https://app.team.cn.test` were issued from a client machine, and the `X-Backend` header was observed to alternate between A and B, confirming round-robin distribution:

```

1 $ curl --cacert ~/app.team.cn.test.crt https://app.team.cn.test
2 {"backend":"A", "message": "Backend A is running"}
3 $ curl --cacert ~/app.team.cn.test.crt https://app.team.cn.test
4 {"backend":"B","status":"ok","message":"Backend B is running"}
5 $ curl --cacert ~/app.team.cn.test.crt https://app.team.cn.test
6 {"backend": "A", "message": "Backend A is running"}
7 $ curl --cacert ~/app.team.cn.test.crt https://app.team.cn.test
8 {"backend":"B","status":"ok","message":"Backend B is running"}
```

Listing 7: Observed alternating responses

Backend-side logs corroborate this: Backend A's access log shows incoming requests originating from Mac 2's IP address (10.7.20.207) rather than the original client's IP, since `nginx` – not the client – makes the connection to the backend. This demonstrates why a client never needs to know the backend servers' IP addresses.



```

Vaageesh@Vaageeshs-MacBook-Pro ~ % curl --cacert ~/app.team.cn.test.crt https://app.team.cn.test
^C
Vaageesh@Vaageeshs-MacBook-Pro ~ % curl --cacert ~/app.team.cn.test.crt https://app.team.cn.test
{"backend":"B","status":"ok","message":"Backend B is running"}
Vaageesh@Vaageeshs-MacBook-Pro ~ % curl --cacert ~/app.team.cn.test.crt https://app.team.cn.test
{"backend": "A", "message": "Backend A is running"}
Vaageesh@Vaageeshs-MacBook-Pro ~ % curl --cacert ~/app.team.cn.test.crt https://app.team.cn.test
{"backend":"B","status":"ok","message":"Backend B is running"}
Vaageesh@Vaageeshs-MacBook-Pro ~ % curl --cacert ~/app.team.cn.test.crt https://app.team.cn.test
{"backend": "A", "message": "Backend A is running"}
Vaageesh@Vaageeshs-MacBook-Pro ~ % curl --cacert ~/app.team.cn.test.crt https://app.team.cn.test
```

Figure 4: Round-robin load balancing: alternating X-Backend responses

```

[ananyanarang@Ananyas-MacBook-Pro-3 backend_a % nano server.py
[ananyanarang@Ananyas-MacBook-Pro-3 backend_a % python3 server.py
Backend A running on http://0.0.0.0:3001
[Backend A] 10.7.10.70 - "GET /api/status HTTP/1.1" 200 -
[Backend A] 10.7.20.207 - "GET / HTTP/1.1" 200 -
[Backend A] 10.7.20.207 - "GET / HTTP/1.1" 200 -
[Backend A] 10.7.20.207 - "GET / HTTP/1.1" 200 -
[Backend A] 10.7.20.207 - "GET / HTTP/1.1" 200 -
[Backend A] 10.7.20.207 - "GET / HTTP/1.1" 200 -
[Backend A] 10.7.20.207 - "GET / HTTP/1.1" 200 -
[Backend A] 10.7.20.207 - "GET / HTTP/1.1" 200 -

```

Figure 5: Backend A access log showing requests originating from nginx's IP (10.7.20.207)

7 Task E: HTTPS / TLS Configuration

nginx on Mac 2 terminates TLS for the domain `app.team.cn.test` using a self-signed certificate generated with OpenSSL. The certificate was distributed to client machines and added to each machine's trust store so that connections succeed without certificate warnings and without using `curl -k`.

```

1 openssl req -x509 -newkey rsa:2048 -nodes \
2   -keyout app.team.cn.test.key \
3   -out app.team.cn.test.crt \
4   -days 365 \
5   -subj "/CN=app.team.cn.test"

```

Listing 8: Certificate generation (OpenSSL)

7.1 TLS Handshake Evidence (Wireshark)

A Wireshark capture filtered on `tls.handshake.type == 1` (Client Hello) confirms multiple TLS 1.3 handshakes initiated from the client (10.7.20.217) to the edge (10.7.20.207) on TCP port 443, with Server Name Indication (SNI) correctly set to `app.team.cn.test`:

Source	Destination	Protocol	Length	Info	
10.7.20.217	10.7.20.207	TLSv1.3	392	Client	Hello (SNI=app.team.cn.test)

The frame detail confirms the TCP three-way handshake precedes the TLS handshake (source port 50516 → destination port 443), and that the TLS record layer encrypts all data following the handshake, which is why HTTP headers and JSON payloads are unreadable in the capture after the `ChangeCipherSpec` message.

8 Task F: HTTP Caching Behaviour

Both backends return a `Cache-Control: max-age=60` header, allowing clients to reuse a cached response for 60 seconds without re-contacting the server.

```

1 curl -I --cacert ~/app.team.cn.test.crt https://app.team.cn.test

```

Listing 9: Verifying cache headers

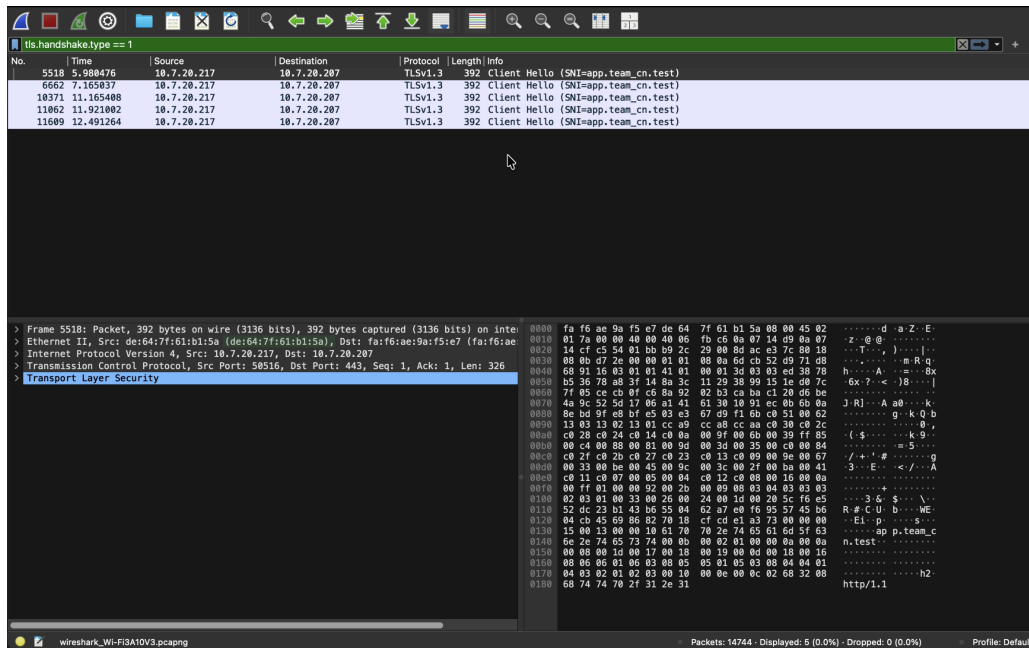


Figure 6: TLS 1.3 Client Hello captured in Wireshark, showing SNI = app.team_cn.test

```

1 HTTP/1.1 200 OK
2 Content-Type: application/json
3 X-Backend: A
4 Cache-Control: max-age=60

```

Listing 10: Relevant response headers

This demonstrates a fresh cache hit within the 60-second window (the client does not need to issue a new request to the server), as distinct from a conditional request (which would use If-None-Match / ETag and receive a 304 Not Modified) and a full new request (issued once the 60-second window expires).

9 Task G: Complete Protocol Flow Capture

Wireshark was used on the client machine to capture and verify every layer of a single request to `app.team.cn.test`.

9.1 DNS Layer

A DNS statistics capture confirms the query/response exchange for the private domain, including query type (A/HTTPS) and round-trip timing for the resolution.

9.2 TCP Handshake

The `SYN` → `SYN-ACK` → `ACK` sequence was captured prior to both the backend HTTP connection (port 3001/3002) and the edge HTTPS connection (port 443), confirming connection establishment before any application data is exchanged.

9.3 TLS Handshake

As shown in Section 5.1, the `ClientHello`, `ServerHello`, `Certificate`, and `ChangeCipherSpec` messages were captured, confirming TLS negotiation and that subsequent application data is encrypted.

9.4 HTTP Headers

`curl -v` output and browser DevTools were used to inspect plaintext request/response headers *before* encryption is applied (i.e., at the application layer, prior to the TLS record layer), and to confirm the `X-Backend` and `Cache-Control` headers appear correctly in every response.

9.5 Port Identification Summary

Layer	Client (ephemeral)	Server (well-known)
DNS	random UDP port	53/UDP (Mac 1)
HTTPS (edge)	random TCP port (e.g., 50516)	443/TCP (Mac 2)
Backend A (internal)	nginx-assigned port	3001/TCP (Mac 3)
Backend B (internal)	nginx-assigned port	3002/TCP (Mac 4)

10 Conclusion and Learning Summary

This project provided hands-on experience with the complete lifecycle of a client request across DNS, TCP, TLS, and HTTP layers using only local infrastructure. Key takeaways for the team include:

- DNS resolution and TCP/TLS connection establishment are independent steps – a name can resolve correctly while the subsequent connection still fails, and vice versa.
- A reverse proxy / load balancer abstracts backend identity from the client entirely; clients only ever need to know the edge’s domain name.
- TLS termination at the edge means internal traffic between the proxy and backend servers is unencrypted, which has implications for Phase 2’s service isolation work.
- `Cache-Control` headers materially reduce redundant requests to the backend, forming the conceptual basis for CDN caching.

This document forms the Phase 1 Architecture Document deliverable and will be extended in Phase 2 to include backup DNS, TTL behaviour, service isolation, and HA failover evidence.